

Домашнее задание: Архитектура React-приложений (максимум — 12 баллов)

Цель

Закрепить принципы архитектурного проектирования React-приложений на основе Feature-Sliced Design (FSD), научиться разделять ответственность между слоями и применять FSD в практическом мини-проекте.

Основная часть (до 10 баллов)

1. Создание проекта по FSD (3 балла)

Задание

Создайте новый проект на React с TypeScript и настройте базовую архитектуру по принципам Feature-Sliced Design (FSD).

Что нужно сделать

- Инициализируйте проект через Vite (`npm create vite@latest`).
- Настройте такую структуру каталогов внутри `src/`:

```
src/  
  app/  
  entities/  
  features/  
  shared/  
  pages/  
  widgets/
```

Что должно находиться в `app/`:

- `App.tsx` — базовый корневой компонент.
 - `router.tsx` — файл с определением маршрутов.
- с. Настройте **ESLint** и **Prettier** с конфигурацией, отражающей принципы FSD: поддержка абсолютных импортов через `tsconfig` paths. Рекомендуется подключить алиасы сразу, чтобы избежать хаоса с относительными путями на следующих этапах. Пример `tsconfig.json` с путями:

```
{
  "compilerOptions": {
    "baseUrl": "src",
    "paths": {
      "shared/*": ["shared/*"],
      "entities/*": ["entities/*"],
      "features/*": ["features/*"],
      "widgets/*": ["widgets/*"],
      "pages/*": ["pages/*"],
      "app/*": ["app/*"]
    }
  }
}
```

Критерии:

- Проект запускается без ошибок — **1 балл**;
- Структура соответствует FSD — **1 балл**;
- Настроены ESLint и Prettier с конфигурацией, отражающей принципы FSD, — **1 балл**.

2. Реализация сущности Task (3 балла)

Задание

Создайте бизнес-сущность **task** в слое **entities/**. Опишите тип, реализуйте UI-компонент для отображения задачи и примените стилизацию через CSS-модули.

Интерфейс задачи:

```
// entities/task/model/types.ts
export interface Task {
  id: string;
  title: string;
  completed: boolean;
```

```
}
```

Компонент отображения:

Создайте компонент `TaskCard.tsx` — он должен быть **презентационным**, то есть:

- принимает данные через `props`;
- не содержит логики или side-effects;
- рендерит заголовок и статус (например, галочка/иконка/цвет);
- использует CSS-модули для стилизации.

Рекомендуемая структура папок:

```
entities/  
  task/  
    model/  
      types.ts  
    ui/  
      TaskCard.tsx  
      TaskCard.module.css
```

Стили для `TaskCard` должны располагаться рядом с компонентом, в файле `TaskCard.module.css`.

Критерии:

- Типизация и структура — **1 балл**.
- Компонент `TaskCard` с правильной презентационной логикой — **1 балл**.
- Стили через `.module.css` — **1 балл**.

3. Создание виджета «Список задач» (3 балла)

Задание

Реализуйте фичу `taskList` в слое `widgets/`. Она должна отображать список задач и обеспечивать интерактивные действия: фильтрацию по статусу и удаление.

Рекомендуемая структура папок

```
widgets/  
  taskList/  
    model/
```

```
useTasks.ts
ui/
  TaskList.tsx
```

Используйте кастомный хук `useTasks`. API хука:

```
// widgets/taskList/model/useTasks.ts
import { Task } from 'entities/task/model/types';

export type Filter = 'all' | 'completed' | 'incomplete';

function useTasks(initial: Task[]): {
  tasks: Task[];           // отфильтрованные задачи
  filter: Filter;           // текущий фильтр
  setFilter: (f: Filter) => void; // смена фильтра
  removeTask: (id: string) => void; // удаление задачи по ID
}
```

В качестве «начальных данных» для этого задания можно использовать локальный массив прямо внутри хука (например, `useState([...])`).

Требования к логике:

- Задачи фильтруются по статусу:
 - `"all"` — все задачи;
 - `"completed"` — только завершённые;
 - `"incomplete"` — только незавершённые.
- При удалении задачи она исчезает из UI без перезагрузки.
- Все действия должны отражаться сразу в интерфейсе.

Критерии:

- Хук корректно реализует фильтрацию и удаление — **1 балл**.
- Компонент `TaskList` правильно отображает отфильтрованные задачи — **1 балл**.
- Используется state-хук и проброс пропсов — **1 балл**.

4. Вывод задач на странице (2 балла)

Задание

- Создайте страницу `TaskPage` в слое `pages/`, которая будет отображать список задач с фильтрацией.

Структура:

```
pages/  
  tasks/  
    ui/  
      TaskPage.tsx
```

- Внутри страницы подключите виджет `TaskWidget`, реализующий визуальную обертку над `TaskList`, фильтрацией и логикой управления задачами.

Пример импорта:

```
// pages/tasks/ui/TaskPage.tsx  
import { TaskWidget } from 'widgets/task';  
  
export function TaskPage() {  
  return (  
    <div>  
      <h1>Мои задачи</h1>  
      <TaskWidget />  
    </div>  
  );  
}
```

Критерии:

- Страница и виджет работают корректно — **1 балл**.
- Читаемый, чистый и модульный код — **1 балл**.

Дополнительная задача (1 бонусный балл)

Эта задача не обязательна, но может помочь набрать баллы до 12.

Вынесите кнопку фильтрации в shared-компонент (1 балл)

- Создайте компонент `FilterButton` в `shared/ui`.
- Используйте его внутри виджета `TaskList`.